

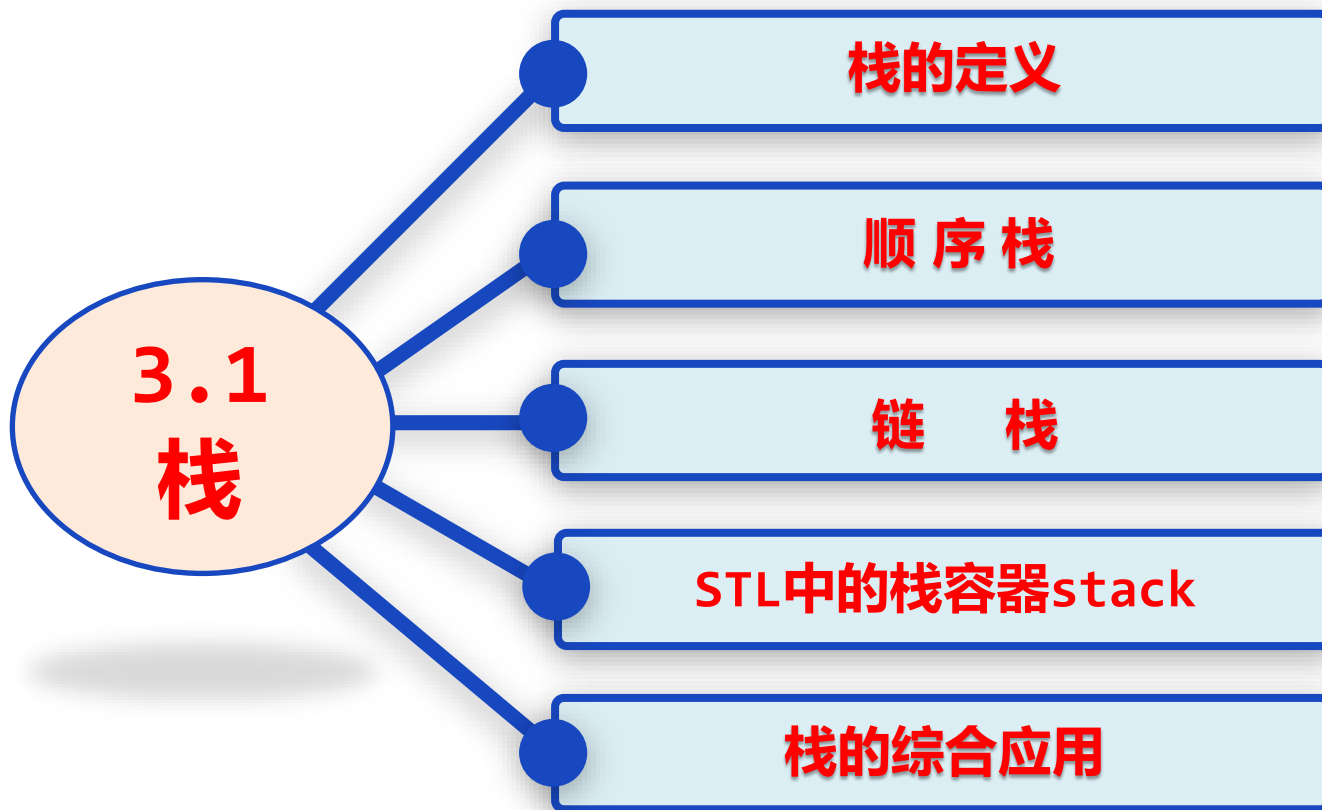
第3章 栈和队列

提纲

CONTENTS

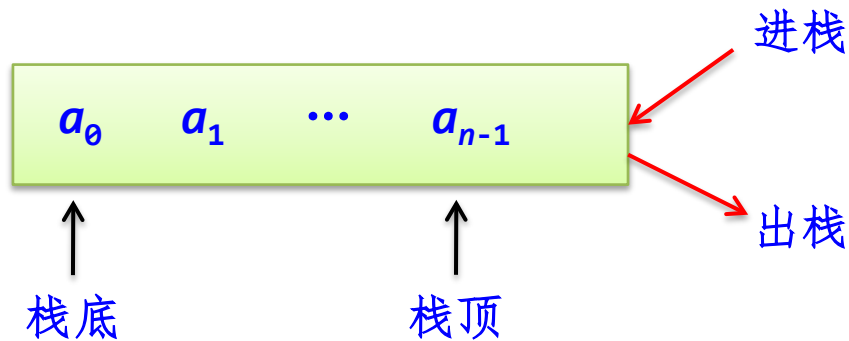
3.1 栈

3.2 队列



3.1.1 栈的定义

- 栈（**stack**）是一种只能在一端进行插入或删除操作的线性表。
- 表中允许进行插入、删除操作的一端称为栈顶（**top**），表的另一端称为栈底（**bottom**）。
- 栈的插入操作通常称为进栈或入栈（**push**），栈的删除操作通常称为退栈或出栈（**pop**）。





后放置的木块先取出来

栈的主要特点:

- 后进先出，即后进栈的元素先出栈。
- 每次进栈的元素都作为新栈顶元素，每次出栈的元素只能是当前栈顶元素。
- 栈也称为后进先出表或者先进后出表。

ADT Stack

{

数据对象:

$D = \{a_i \mid 0 \leq i \leq n-1, n \geq 0, \text{元素 } a_i \text{ 为 } T \text{ 类型}\}$

数据关系:

$R = \{r\}$

$r = \{\langle a_i, a_{i+1} \rangle \mid a_i, a_{i+1} \in D, i = 0, \dots, n-2\}$

基本运算:

`empty()`: 判断栈是否为空, 若空栈返回真; 否则返回假。

`push(T e)`: 进栈操作, 将元素 `e` 插入到栈中作为栈顶元素。

`pop(T& e)`: 出栈操作。

`gettop(T& e)`: 取栈顶操作。

}

栈抽象数据类型 = 线性结构 + 栈的基本运算

示例

一个栈的进栈序列是a、b、c、d、e，则栈的不可能的输出序列是（ ）。

A.edcba

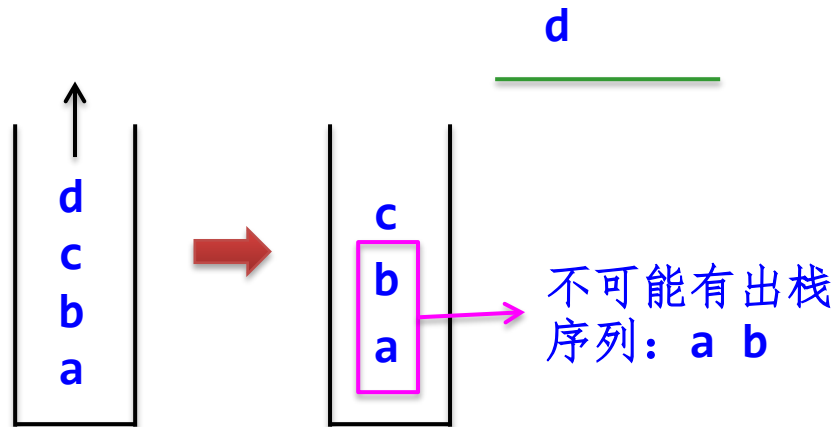
B.decba

C.dceab

D.abcde

用栈模拟进行判断

考虑 C.dceab



示例

已知一个栈的进栈序列是 $1, 2, 3, \dots, n$ ，其输出序列是 $p_1, p_2, \dots, p_n$ ，若 $p_1=n$ ，则 p_i 的值为（ ）。

A. i

B. $n-i$

C. $n-i+1$

D. 不确定

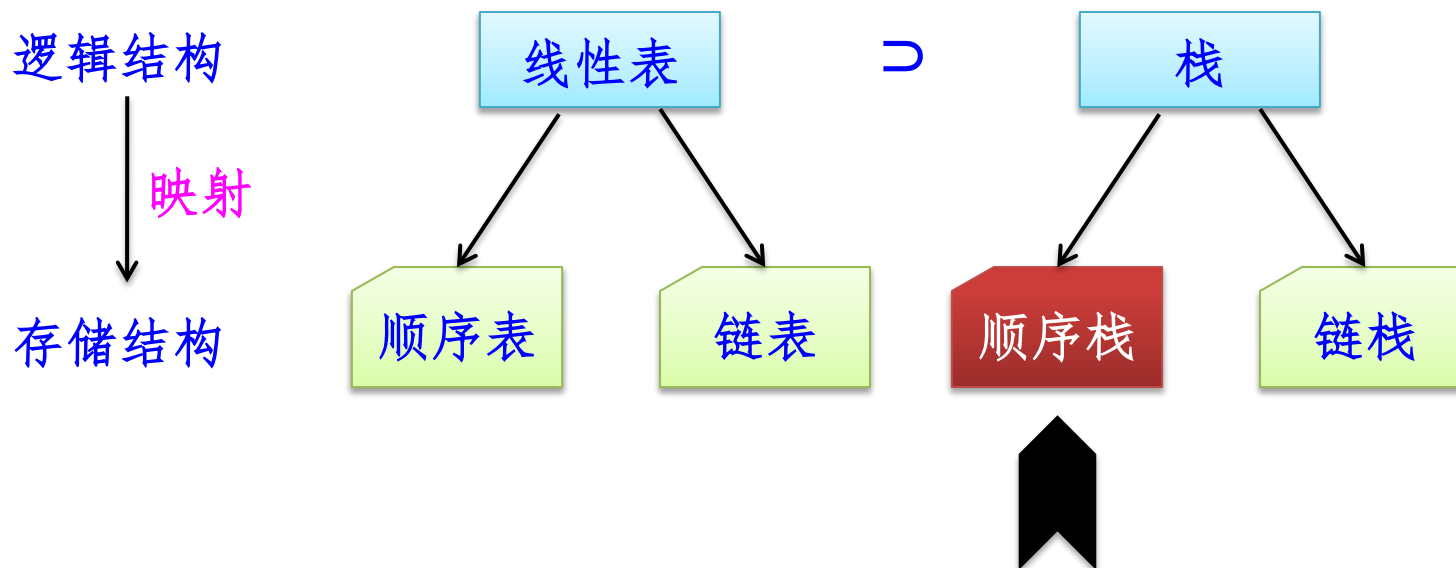
$1, 2, 3, \dots, n \longrightarrow n, \dots$

输出序列唯一

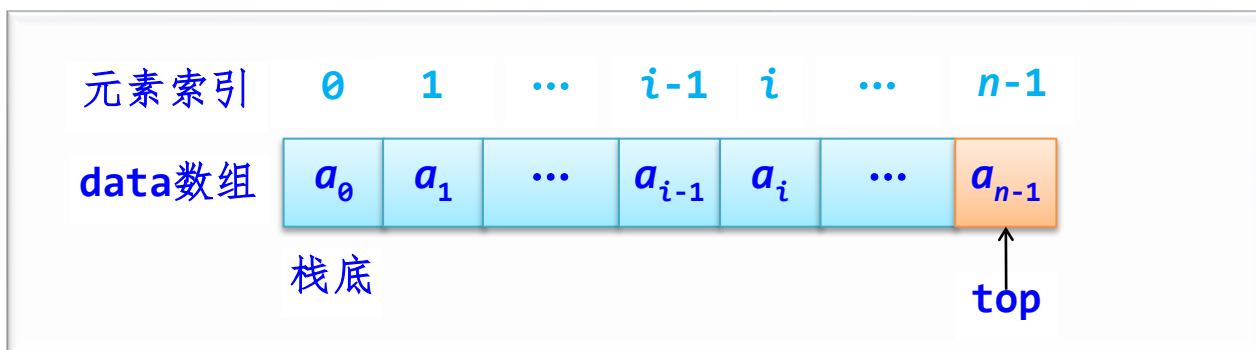
$$\left. \begin{array}{l} p_1=n \\ p_2=n-1 \\ p_3=n-2 \\ \dots \\ p_n=1 \end{array} \right\} p_i+i=n+1 \text{ 即 } p_i=n-i+1$$

3.1.2 栈的顺序存储结构及其基本运算算法实现

栈的实现方式

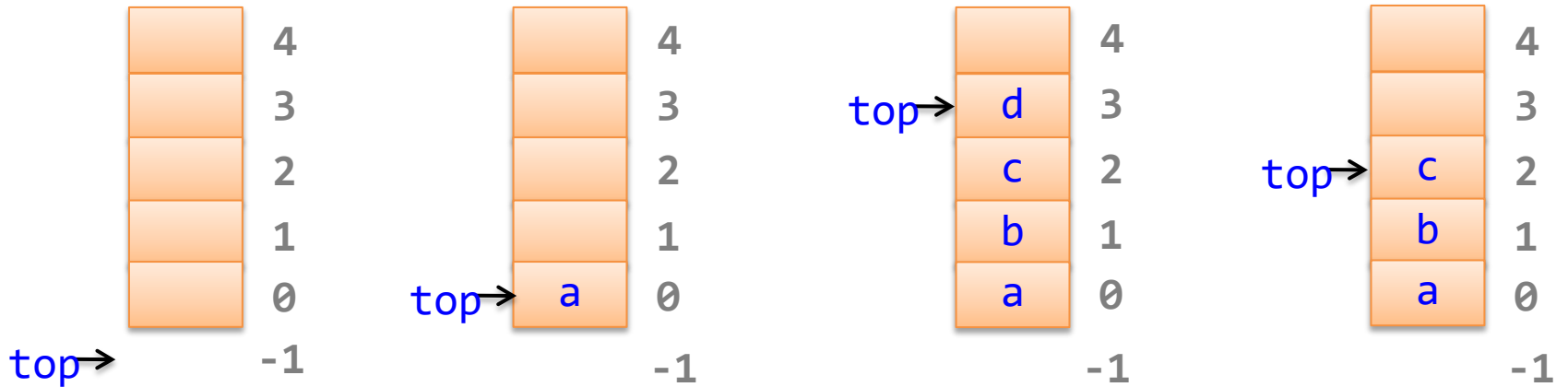


顺序栈实现



- 由于栈顶是动态变化的，为此设置一个栈顶指针`top`以反映栈状态，约定`top`总是指向栈顶元素。
- 为了简单，这里的`data`数组采用固定容量（容量为`MaxSize`）分配方式，并且置`data[0]`端作为栈底，另外一端`data[MaxSize-1]`作为栈顶，其中的元素个数恰好为`top+1`。

顺序栈 (MaxSize=5)



(a) 空栈 (b) 元素a进栈 (c) 元素b、c、d进栈 (d) 元素d出栈

顺序栈的四要素如下（初始时`top=-1`）：

- ① 栈空条件: `top == -1`。
- ② 栈满条件: `top == MaxSize - 1`。
- ③ 元素`e`进栈操作: `top++`; `data[top] = e`。
- ④ 出栈操作: `e = data[top]`; `top--`。

顺序栈类模板SqStack

```
template <typename T>
class SqStack           //顺序栈类模板
{   T* data;           //存放栈中元素
    int top;           //栈顶指针
    //栈的基本运算算法
};
```

顺序栈的基本运算算法

(1) 顺序栈的初始化和销毁

```
SqStack()  
{  
    data=new T[MaxSize];  
    top=-1;  
}
```

//构造函数
//为data分配容量为MaxSize的空间
//栈顶指针初始化

```
~SqStack()  
{  
    delete [] data;  
}
```

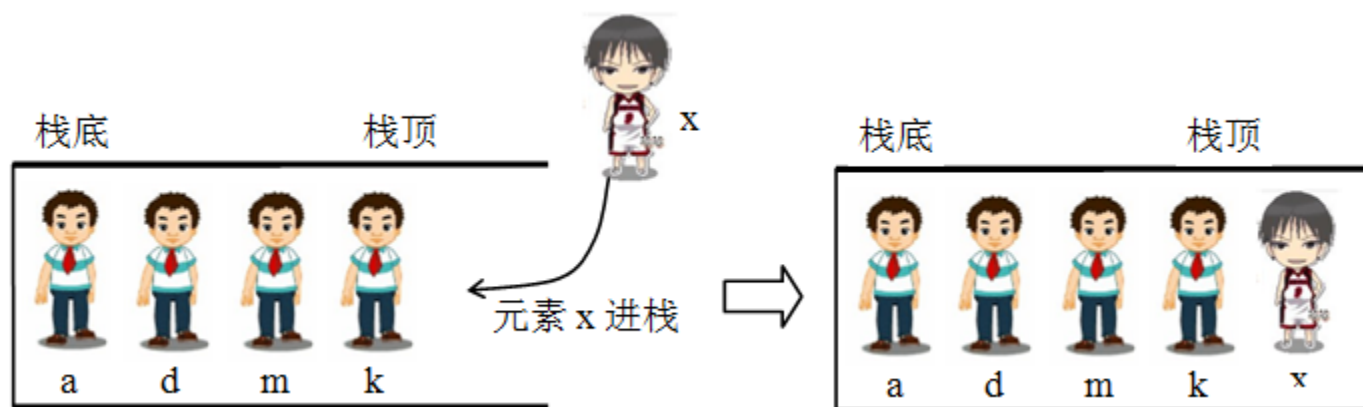
//析构函数

(2) 判断栈是否为空empty()

```
bool empty()           //判断栈是否为空
{
    return top==-1;
}
```

(3) 进栈push(e)

元素进栈只能从栈顶进，不能从栈底或中间位置进栈

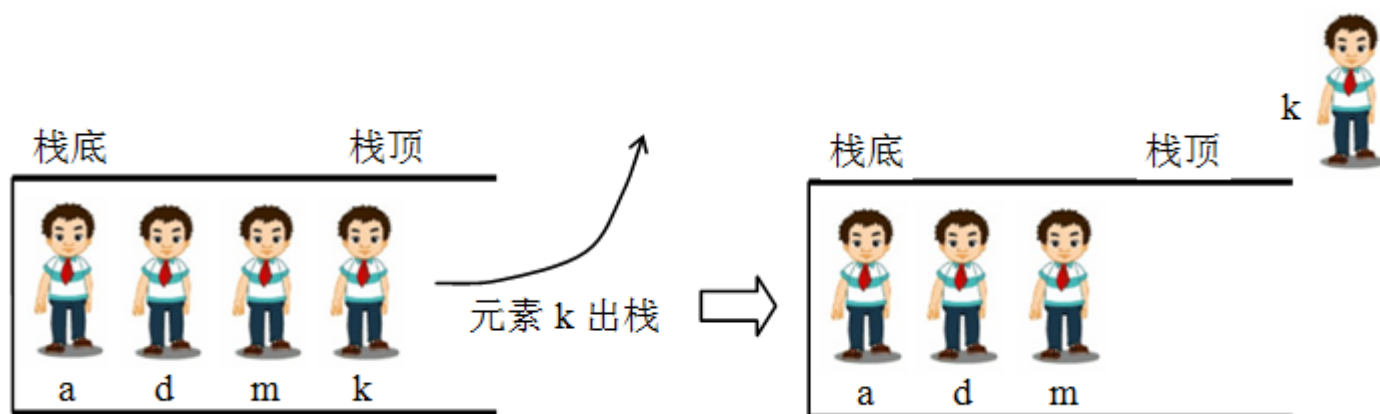


```
bool push(T e)           //进栈算法
{ if (top==MaxSize-1)    //栈满时返回false
    return false;

    top++;               //栈顶指针增1
    data[top]=e;         //将e进栈
    return true;
}
```


(4) 出栈pop()

元素出栈只能从栈顶出，不能从栈底或中间位置出栈



```
bool pop(T& e)
{  if (empty())
    return false;
    e=data[top];
    top--;
    return true;
}
```

//出栈算法

//栈为空的情况，即栈下溢出

//取栈顶指针元素的元素

//栈顶指针减1

(5) 取栈顶元素gettop()

```
bool gettop(T& e)
{  if (empty())
    return false;
    e=data[top];
    return true;
}
```

//取栈顶元素算法

//栈为空的情况，即栈下溢出

//取栈顶指针位置的元素

3.1.3 顺序栈的应用算法设计示例

【例3.4】 设计一个算法利用顺序栈检查用户输入的表达式中括号是否配对（假设表达式中可能含有圆括号、中括号和大括号）。并用相关数据进行测试。



```
#include "SqStack.cpp"  
bool isMatch(string str)
```

```
{ SqStack<char> st;
```

```
  int i=0;
```

```
  char e;
```

```
  while (i<str.length())
```

```
  {  if (str[i]=='(' || str[i]=='[' || str[i]=='{')
```

```
      st.push(str[i]);    //遇到将左括号,均进栈
```

//包含顺序栈类模板的定义

//判断表达式各种括号是否匹配的算法

//建立一个顺序栈

```

else
{
    if (str[i]=='')
    {
        if (st.empty())
            return false;
        st.pop(e);
        if (e!='(')
            return false;
    }
    if (str[i]==']')
    {
        if (st.empty())
            return false;
        st.pop(e);
        if (e!='[')
            return false;
    }
    if (str[i]=='}')
    {
        if (st.empty())
            return false;
        st.pop(e);
        if (e!='{')
            return false;
    }
    }
    i++;
}
return st.empty();
}

```

```

//遇到')'
//栈空时返回false

//出栈元素e
//栈顶不是匹配的'(',返回false

//遇到']'
//栈空时返回false

//出栈元素e
//栈顶不是匹配的'[',返回false

//遇到'}'
//栈空时返回false

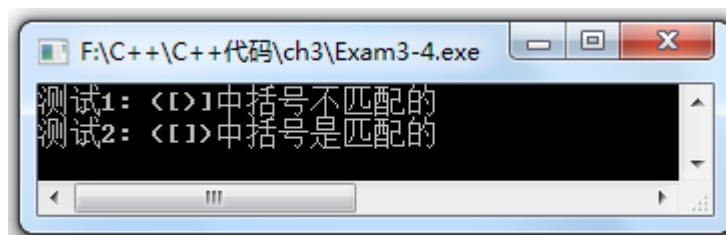
//出栈元素e
//栈顶不是匹配的'{',返回false

//继续遍历str

```

```
int main()
{  cout << "测试1: ";
   string str="([])";
   if (isMatch(str))
       cout << str << "中括号是匹配的" << endl;
   else
       cout << str << "中括号不匹配的" << endl;

   cout << "测试2:";
   str="([])";
   if (isMatch(str))
       cout << str << "中括号是匹配的" << endl;
   else
       cout << str << "中括号不匹配的" << endl;
   return 0;
}
```



共享栈问题

【例3.7】设有两个栈S1和S2，它们都采用顺序栈存储，并且共享一个固定容量的存储区 $s[0..M-1]$ ，为了尽量利用空间，减少溢出的可能，请设计这两个栈的存储方式。



为了尽量利用空间，减少溢出的可能，可以让两个的栈顶相向即进栈元素迎面增长的存储方式，为此设置两个栈的栈顶指针分别为 $top1$ 和 $top2$ （均指向对应栈的栈顶元素）。



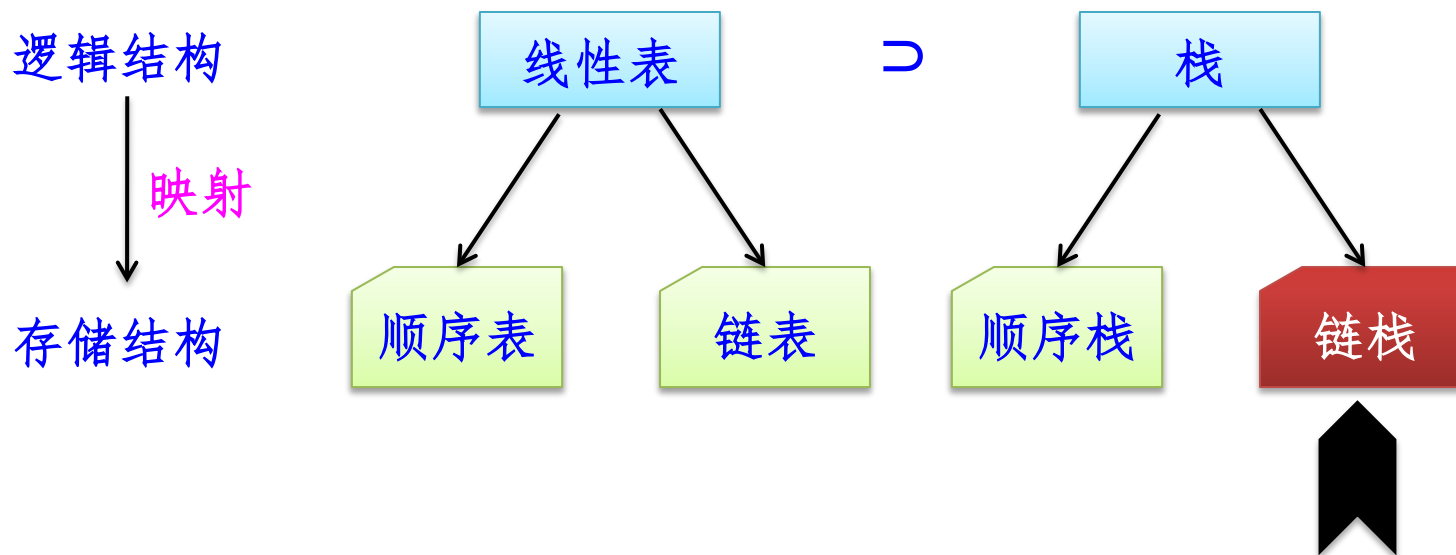


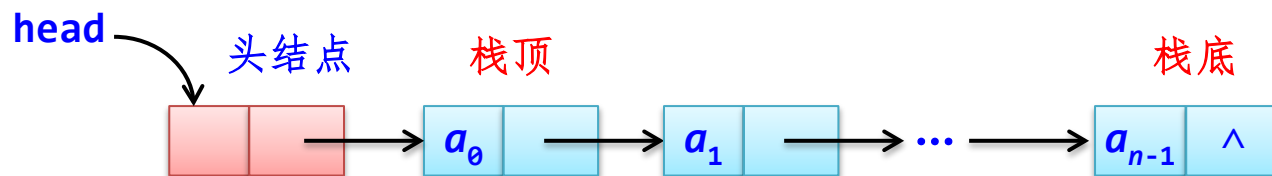
- 栈S1空的条件是 $top1 = -1$ 。
- 栈S1满的条件是 $top1 = top2 - 1$;
- 元素 e 进栈S1（栈不满时）的操作是： $top1++; s[top1] = e$ 。
- 元素 e 出栈S1（栈不空时）的操作是： $e = s[top1]; top1--$ 。

- 栈S2空的条件是 $top2 = M$ 。
- 栈S2满的条件是 $top2 = top1 + 1$ 。
- 元素 e 进栈S2（栈不满时）的操作是： $top2--; s[top2] = e$ 。
- 元素 e 出栈S2（栈不空时）的操作是： $e = s[top2]; top2++$ 。

3.1.4 栈的链式存储结构及其基本运算算法实现

栈的实现方式





初始时只含有一个头结点head并置head->next为NULL。这样链栈的四要素如下：

- 栈空的条件：head->next==NULL。
- 由于只有在内存溢出才会出现栈满，通常不考虑这种情况。
- 元素 e 进栈操作：将包含该元素的结点 s 插入作为首结点。
- 出栈操作：返回首结点值并且删除该结点。

和单链表一样，链栈中每个结点的类型**LinkNode**如下

```
template <typename T>
struct LinkNode                                //链栈结点类型
{
    T data;                                     //数据域
    LinkNode* next;                             //指针域
    LinkNode():next(NULL) {}                   //构造函数
    LinkNode(T d):data(d),next(NULL) {}        //重载构造函数
};
```

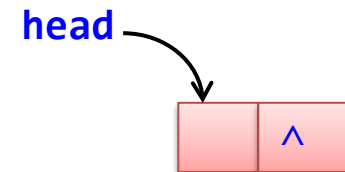
链栈类模板LinkStack

```
template <typename T>
class LinkStack           //链栈类模板
{
public:
    LinkNode<T>* head;    //链栈头结点
    #栈的基本运算算法
};
```

链栈的基本运算算法

(1) 链栈的初始化和销毁

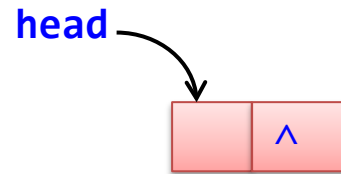
```
LinkStack()           //构造函数
{
    head=new LinkNode<T>();
}
```



```
~LinkStack()          //析构函数
{
    LinkNode<T>* pre=head,*p=pre->next;
    while (p!=NULL)
    {
        delete pre;
        pre=p; p=p->next;    //pre、p同步后移
    }
    delete pre;
}
```

(2) 判断栈是否为空empty()

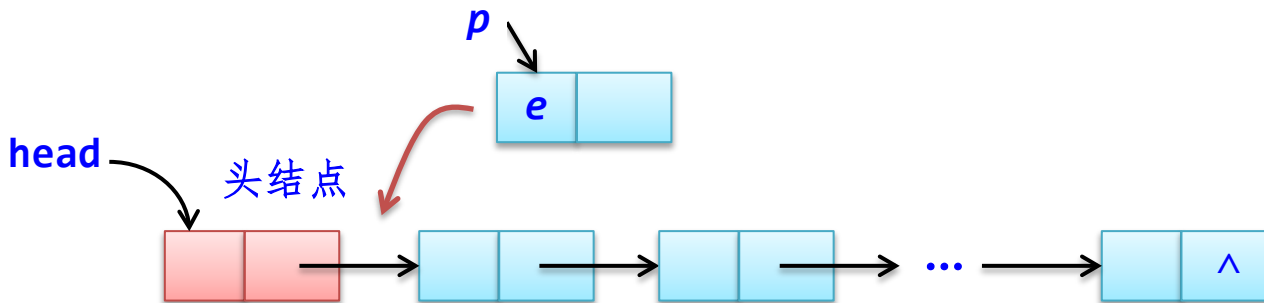
```
bool empty() //判断栈是否为空
{
    return head->next==NULL;
}
```



(3) 进栈push(e)

```
bool push(T e)
{
    LinkNode<T>* p=new LinkNode<T>(e);
    p->next=head->next;
    head->next=p;
    return true;
}
```

//进栈算法
//新建结点p
//插入结点p作为首结点



(4) 出栈pop()

```
bool pop(T& e)
{
    LinkNode<T>* p;
    if (head->next==NULL)
        return false;
    p=head->next;
    e=p->data;
    head->next=p->next;
    delete p;
    return true;
}
```

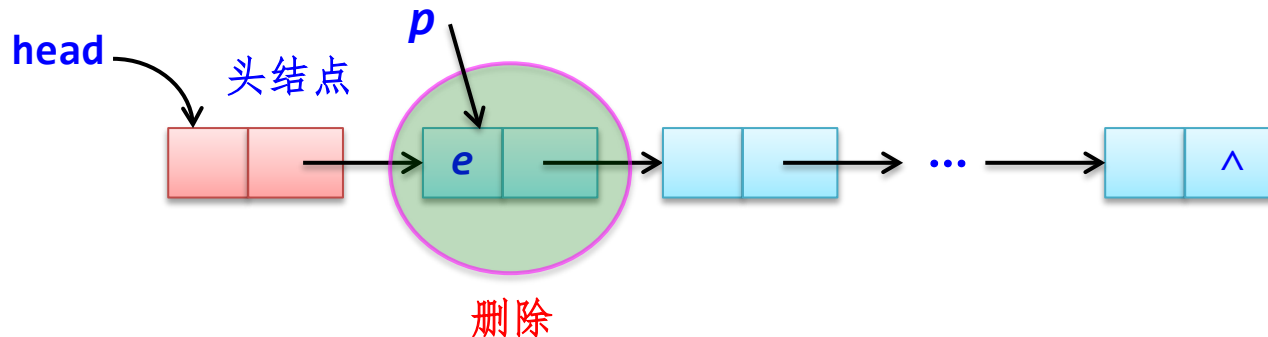
//出栈算法

//栈空的情况

//p指向开始结点

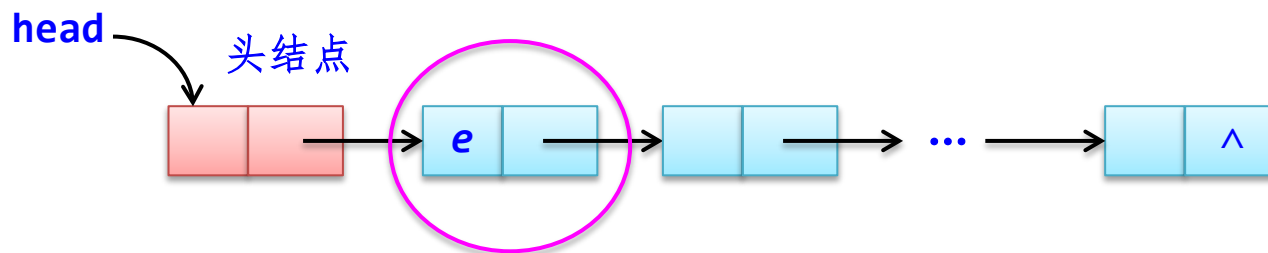
//删除结点p

//释放结点p



(5) 取栈顶元素gettop()

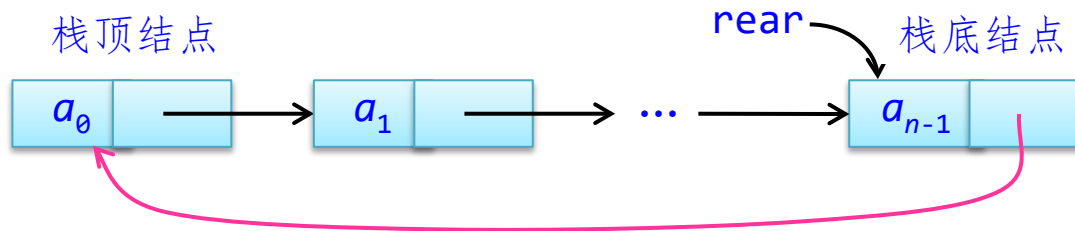
```
bool gettop(T& e)           //取栈顶元素
{   LinkNode<T>* p;         //栈空的情况
    if (head->next==NULL)
        return false;
    p=head->next;            //p指向开始结点
    e=p->data;
    return true;
}
```

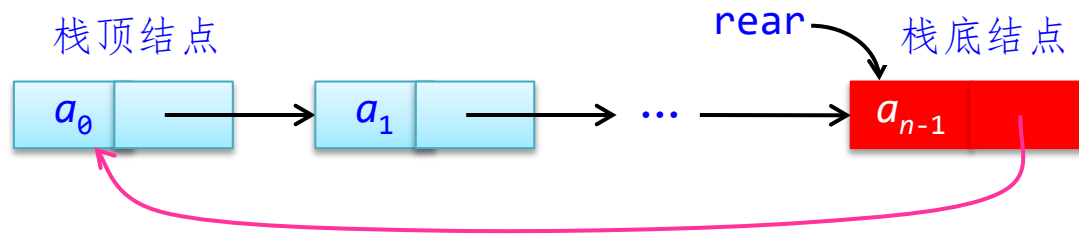


3.1.5 链栈的应用算法设计示例

【例3.9】 定义一个栈数据结构STACK，添加一个Getbottom()运算用于直接返回栈底元素（假设栈不空时）。要求采用链表实现并且函数Getbottom()、push以及pop的时间复杂度都是 $O(1)$ 。

- 如果采用普通单链表实现，以前端为栈顶后端为栈底，那么找到尾结点（存放栈底元素）的时间复杂度为 $O(n)$ ，不满足题目要求。
- 改为不带头结点仅有尾结点指针的循环单链表 $rear$ 作为链栈。





初始时 $\text{rear}=\text{NULL}$ ，栈的四要素如下：

- ① 栈空条件： $\text{rear}=\text{NULL}$ 。
- ② 栈满条件：不考虑。
- ③ 元素 e 进栈操作：建立含 e 元素的结点 p ，将结点 p 插入到 rear 结点的后面。
- ④ 元素 e 出栈操作：取结点 rear 之后的结点值 e ，删除该结点。

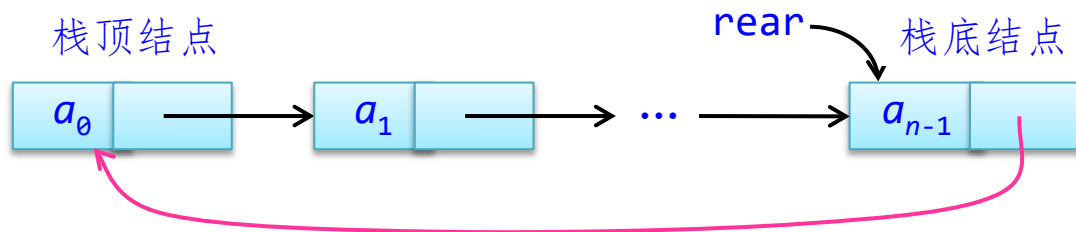
Getbottom()函数就是返回rear结点值即 $\text{rear} \rightarrow \text{data}$ （栈不空时）

```

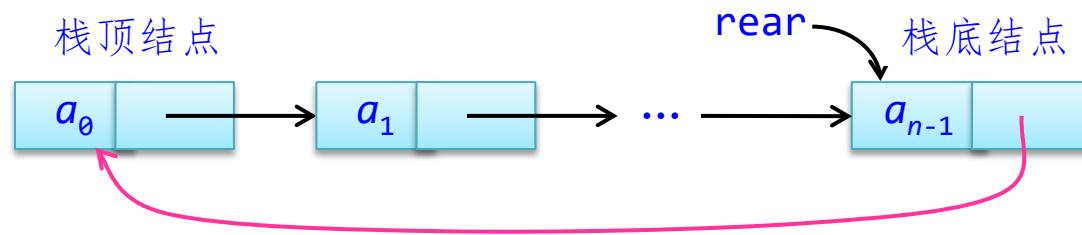
template <typename T>
struct LinkNode                                     //链栈结点类型
{
    T data;                                         //数据域
    LinkNode* next;                               //指针域

    LinkNode():next(NULL) {}                      //构造函数
    LinkNode(T d):data(d),next(NULL) {}           //重载构造函数
};

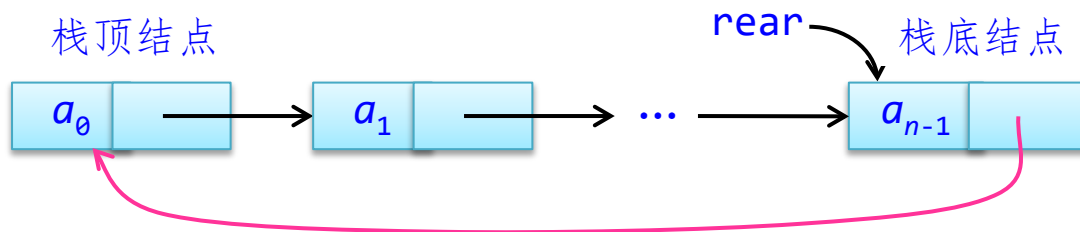
```



template <typename T>	
class STACK	//链栈类模板
{	
public:	
LinkNode<T>* rear;	//链栈尾结点指针
STACK() :rear(NULL) {}	//构造函数
~ STACK()	//析构函数
{ if (rear==NULL) return;	//空链表直接返回
LinkNode<T>* pre=rear,*p=pre->next;	
while (p!=rear)	
{ delete pre;	
pre=p; p=p->next;	//pre、p同步后移
}	
delete pre;	
}	
 bool empty()	//判栈空算法
{	
return rear==NULL;	
}	



```
bool push(T e)                                //进栈算法
{  LinkNode<T>* p=new LinkNode<T>(e);        //新建结点p
  if (empty())                                //栈为空的情况
  {  rear=p;
    rear->next=rear;
  }
  else                                         //栈不空的情况
  {  p->next=rear->next;                      //将结点p插入到结点rear之后
    rear->next=p;
  }
  return true;
}
```

```

bool pop(T& e)
{  LinkNode<T>* p;
   if (empty()) return false;
   if (rear->next==rear)
   {  p=rear;
      rear=NULL;
   }
   else
   {  p=rear->next;
      rear->next=p->next;
   }
   e=p->data;
   delete p;
   return true;
}

```

//出栈算法

//栈空的情况

//栈中只有一个结点的情况

//栈中有2个及以上结点的情况

//释放结点p

```

bool gettop(T& e)
{  if (empty()) return false;
    e=rear->next->data;
    return true;
}

```

//取栈顶元素
//栈空的情况

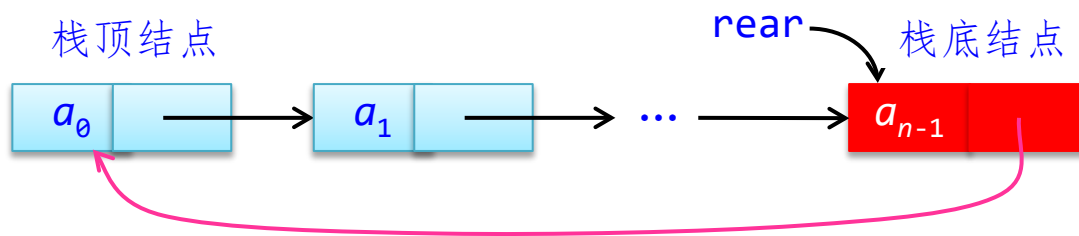
```

T Getbottom()
{
    return rear->data;
}

};

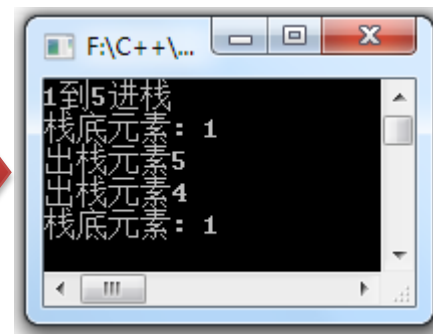
```

//取栈底元素



程序 验证

```
int main()
{   int e;
    STACK<int> st;
    cout << "1到5进栈" << endl;
    for (int i=1;i<=5;i++)
        st.push(i);
    cout << "栈底元素: " << st.Getbottom() << endl;
    st.pop(e);
    cout << "出栈元素" << e << endl;
    st.pop(e);
    cout << "出栈元素" << e << endl;
    cout << "栈底元素: " << st.Getbottom() << endl;
    return 0;
}
```



3.1.6 STL中的栈容器

stack容器主要的成员函数

成员函数	说明
<code>empty()</code>	判断栈是否为空
<code>size()</code>	返回栈中的实际元素个数
<code>push(e)</code>	即元素 <code>e</code> 进栈
<code>top()</code>	返回栈顶元素，当栈空时抛出异常
<code>pop()</code>	出栈一个元素，并不返回出栈的元素，当栈空时抛出异常

提示

- 与前面自己实现的栈运算相比，**stack**容器的成员函数使用更加简单方便。
- 需要注意的是**stack**容器具有空间动态扩展功能，**push()**不会出现上溢出的情况，另外在使用**top()**和**pop()**之前应保证栈不空。

```
#include<iostream>
#include <stack>
using namespace std;
int main()
{   stack<int> st;
    st.push(1); st.push(2); st.push(3);
    printf("栈顶元素: %d\n",st.top());
    printf("出栈顺序: ");
    while (!st.empty())           //栈不空时出栈所有元素
    {   printf("%d ",st.top());
        st.pop() ;
    }
    printf("\n");
    return 0;
}
```



栈顶元素: 3
出栈顺序: 3 2 1

3.1.7 栈的综合应用

求解问题中需要临时保存一些数据元素：

- 先保存的后处理：栈
- 先保存的先处理：队列

简单表达式求值

exp: 仅包含“+”、“-”、“*”、“/”、正整数和小括号的合法数学表达式—**中缀表达式**。

后缀表达式: 就是运算符在操作数的后面，已经考虑了运算符的优先级，不包含括号，只含操作数和运算符。

1+2*3



123*+

exp求值过程

- 将表达式exp转换成后缀表达式postexp。
- 对该后缀表达式求值。

设计求表达式值的类Express

```
class Express
{
    string exp;
    string postexp;
public:
    Express(string str)
    {
        exp=str;
        postexp="";
    }
    string getpostexp()
    {
        return postexp;
    }
    void Trans() { ... }
    double GetValue() { ... }
};
```

```
//求表达式值类
//存放中缀表达式
//存放后缀表达式
```

```
//构造函数
```

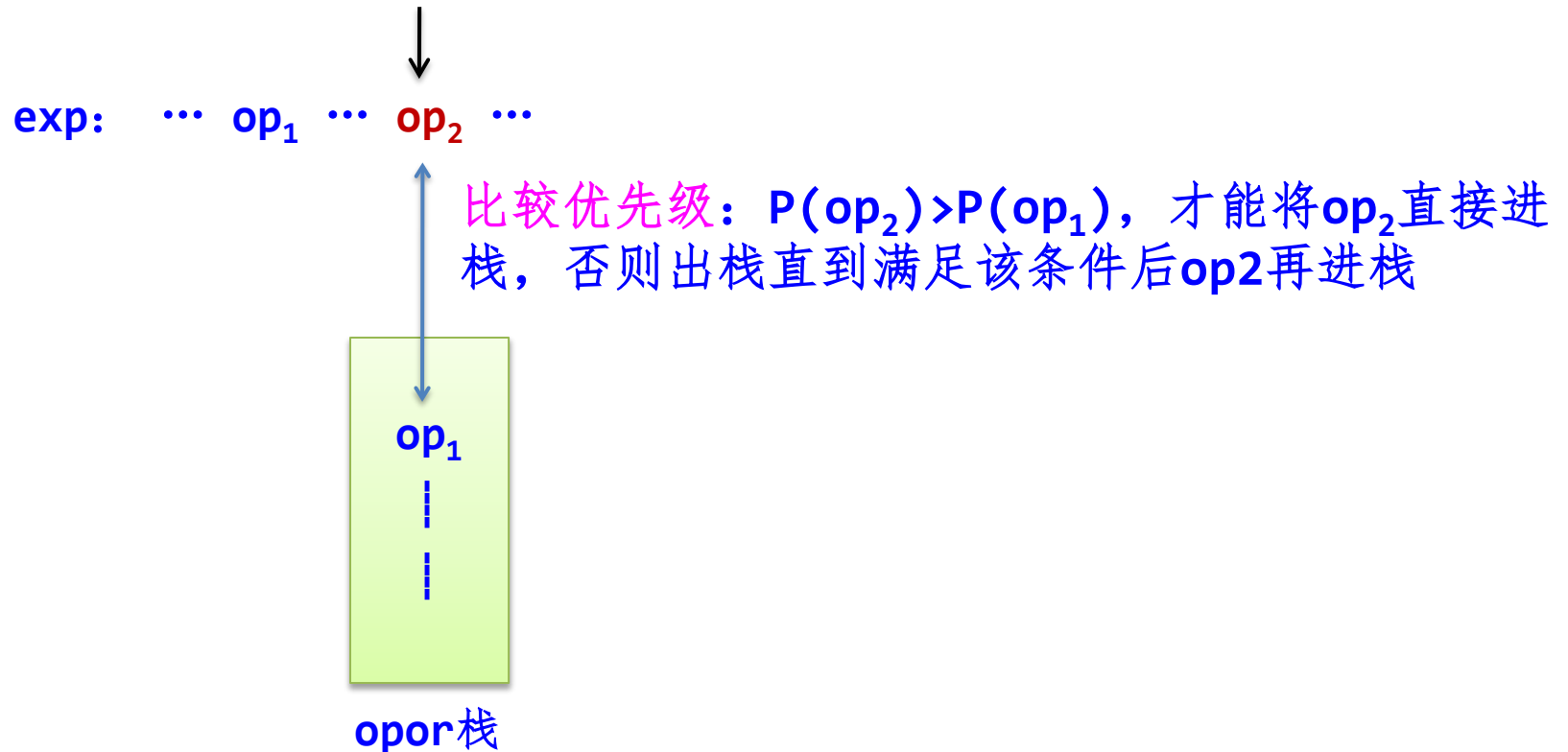
```
//返回postexp
```

```
//将exp转换成postexp
```

```
//计算后缀表达式postexp的值
```

exp $\Rightarrow$ postexp

使用运算符栈**opor**：先出栈的运算符先做运算！



括号的特殊性!

转换过程

while (若exp未读完)

{ 从exp读取字符ch;

ch为数字: 将后续的所有数字均依次存放到postexp中;

ch为左括号'(': 将'('进栈到opor;

ch为右括号')': 将opor栈中'('以前的运算符依次出栈并存放到postexp中,再将'('退栈;

若ch的优先级高于栈顶运算符优先级,则将ch进栈; 否则出栈并存放到postexp中直到该条件成立,再将ch进oper栈;

}

字符串exp扫描完毕,则退栈opor的所有运算符并存放到postexp中

表达式“(56-20)/(4+2)”转换成后缀表达式的过程

ch	操 作	postexp	opor栈
(	将 '(' 进opor栈		(
56	将56存入postexp中，并插入一个字符'#'	"56#"	(
-	由于opor中 '(' 以前没有字符，则直接将 '-' 进opor栈	"56#"	(-
20	将20#存入postexp中	"56#20#"	(-
)	将栈opor中 '(' 以前的运算符依次出栈并存入postexp，然后将 '(' 出栈	"56#20#-"	
/	将 '/' 进opor栈	"56#20#-"	/
(	将 '(' 进opor栈	"56#20#-"	/(
4	将4#存入postexp	"56#20#-4#"	/(
+	由于opor中 '(' 以前没有运算符，则直接将 '+' 进opor栈	"56#20#-4#"	/(+
2	将2#存入postexp	"56#20#-4#2#"	/(+
)	将opor栈中 '(' 以前的运算符依次出栈并存入postexp，然后将 '(' 出栈	"56#20#-4#2#+"	/
	exp扫描完毕，将opor栈中所有运算符出栈并存入postexp，得到最后的后缀表达式	"56#20#-4#2#+/"	

void Trans()	//将算术表达式 exp 转换成后缀表达式 postexp
{ stack<char> opor;	//定义 运算符栈opor
int i=0;	//i为 exp 的下标
char ch,e;	
while (i<exp.length())	// exp 表达式未扫描完时循环
{ ch=exp[i];	
if (ch=='(')	//遇到左括号
opor.push(ch);	//将左括号直接进栈
else if (ch==')')	//遇到右括号
{ while (!opor.empty() && opor.top()!='(')	
{ e=opor.top();	//将栈中 '(' 之前的运算符退栈并存入 postexp
opor.pop();	
postexp+=e;	
}	
opor.pop();	//将(退栈
}	

```

else if (ch=='+' || ch=='-')           //遇到加或减号
{
    while (!opor.empty() && opor.top()!='(')
    {
        e=opor.top();                 //将栈中(之前的所有运算符退栈
        opor.pop();                   //并存入postexp
        postexp+=e;
    }
    opor.push(ch);                     //再将'+'或'-'进栈
}

else if (ch=='*' || ch=='/')           //遇到'*'或'/'号
{
    while (!opor.empty() && opor.top()!='(' &&
           (opor.top()=='*' || opor.top()=='/'))
    {
        e=opor.top();                 //将栈中(之前的所有*或/依次出栈
        opor.pop();                   //并存入postexp
        postexp+=e;
    }
    opor.push(ch);                     //再将'*'或'/'进栈
}

```

```

else
{
    string d="";
    while (ch>='0' && ch<='9')
    {
        d+=ch;
        i++;
        if (i<exp.length())
            ch=exp[i];
        else
            break;
    }
    i--;
    postexp+=d;
    postexp+="#";
}
i++;
}

```

//遇到数字字符

//遇到数字

//提取所有连续的数字字符

//exp没有遍历完时取下一个字符ch

//exp遍历完毕时退出数字判断

//退一个字符

//将数字串存入postexp

//用#标识一个数字串结束

//继续处理其他字符


```
while (!opor.empty())  
{   e=opor.top();  
    opor.pop();  
    postexp+=e;  
}  
}
```

//此时exp扫描完毕,栈不空时循环

//将栈中所有运算符退栈并放入postexp

后缀表达式postexp求值

使用运算数栈opand

while (若postexp未读完)

{ 从postexp读取字符ch;

ch为 '+': 从opand栈出栈两个数值a和b, 计算 $c=b+a$; 将c进栈opand;

ch为 '-': 从opand栈出栈两个数值a和b, 计算 $c=b-a$; 将c进栈opand;

ch为 '*': 从opand栈出栈两个数值a和b, 计算 $c=b*a$; 将c进栈opand;

ch为 '/': 从opand栈出栈两个数值a和b, 若a不为零, 计算 $c=b/a$; 将c进栈opand;

ch为数字字符: 将连续的数字串转换成数值d, 将d进栈opand;

}

opand栈中唯一的数值即为表达式值

后缀表达式"56#20#-4#2#+/"的求值过程

ch序列	说明	st栈
56	遇到56，将56进栈	56
20	遇到20，将20进栈	56, 20
'-'	遇到'-', 出栈两次，将 $56-20=36$ 进栈	36
4	遇到4，将4进栈	36, 4
2	遇到2，将2进栈	36, 4, 2
'+'	遇到'+', 出栈两次，将 $4+2=6$ 进栈	36, 6
'/'	遇到 '/', 出栈两次，将 $36/6=6$ 进栈	6
	postexp扫描完毕，算法结束，栈顶数值6即为所求	

double GetValue()	//计算后缀表达式postexp的值
{ stack<double> opand;	//定义运算数栈opand
double a,b,c,d;	
char ch;	
int i=0;	
while (i<postexp.length())	//postexp字符串未扫描完时循环
{ ch=postexp[i];	
switch (ch)	
{	
case '+':	//遇到+
 a=opand.top(); opand.pop();	//退栈运算数a
 b=opand.top(); opand.pop();	//退栈运算数b
 c=b+a;	//计算c
 opand.push(c);	//将计算结果进栈
break;	

case '-' :

```
a=opand.top(); opand.pop();  
b=opand.top(); opand.pop();  
c=b-a;  
opand.push(c);  
break;
```

```
//遇到-  
//退栈运算数a  
//退栈运算数b  
//计算c  
//将计算结果进栈
```

case '*' :

```
a=opand.top(); opand.pop();  
b=opand.top(); opand.pop();  
c=b*a;  
opand.push(c);  
break;
```

```
//遇到*  
//退栈运算数a  
//退栈运算数b  
//计算c  
//将计算结果进栈
```

case '/' :

```
a=opand.top(); opand.pop();  
b=opand.top(); opand.pop();  
c=b/a;  
opand.push(c);  
break;
```

```
//遇到/  
//退栈运算数a  
//退栈运算数b  
//计算c  
//将计算结果进栈
```

```

default:                                //遇到数字字符
    d=0;                                //将连续的数字字符转换成数值存放到d中
    while (ch>='0' && ch<='9')
    {  d=10*d+(ch-'0');
        i++;
        ch=postexp[i];
    }
    opand.push(d);                        //将数值d进栈
    break;
}
i++;                                    //继续处理其他字符
}
return opand.top();                      //栈顶元素即为求值结果
}

```

```
int main()
{ string str="(56-20)/(4+2)";
  Express obj(str);
  cout << "中缀表达式: " << str << endl;
  cout << "中缀转换为后缀" << endl;
  obj.Trans();
  cout << "后缀表达式: " << obj.getpostexp() << endl;
  cout << "求后缀表达式值" << endl;
  cout << "求值结果:   " << obj.GetValue() << endl;
  return 0;
}
```

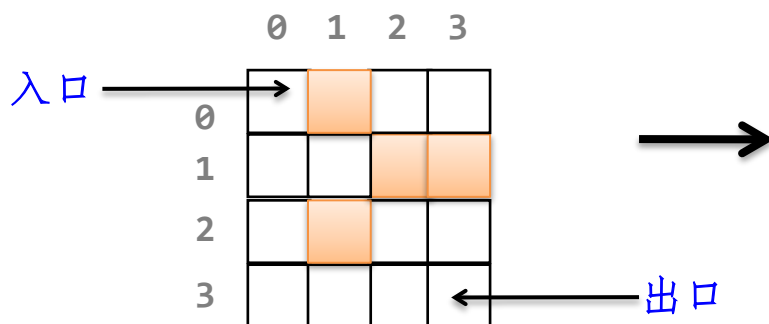


```
F:\C++\C++代码\ch3\Exp...
中缀表达式: <56-20>/<4+2>
中缀转换为后缀
后缀表达式: 56#20#-4#2#+/
求后缀表达式值
求值结果: 6
```

求表达式值的两个步骤可以合并起来，不必产生后缀表达式：

- 将所有遇到的运算数进opand栈。
- 在opor出栈一个运算符op时，从opand中依次出栈两个运算数 a 、 b ，执行 $c = b \text{ op } a$ 运算，将 c 进opand栈。

求迷宫问题



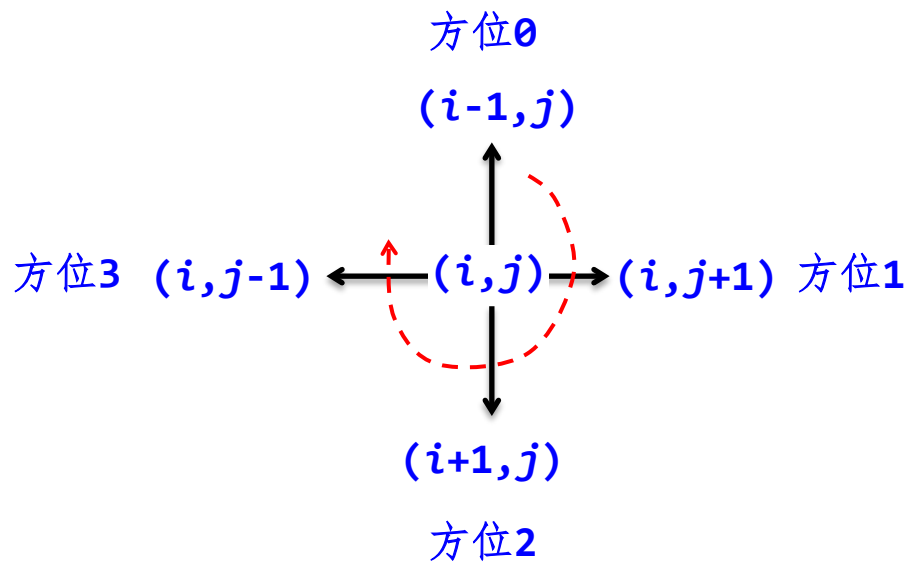
一个迷宫图



求从入口到出口的一条简单路径

```
int mg[MAX][MAX]=  
{0,1,0,0},  
 {0,0,1,1},  
 {0,1,0,0},  
 {0,0,0,0}  
};  
int m=4,n=4;
```

试探顺序

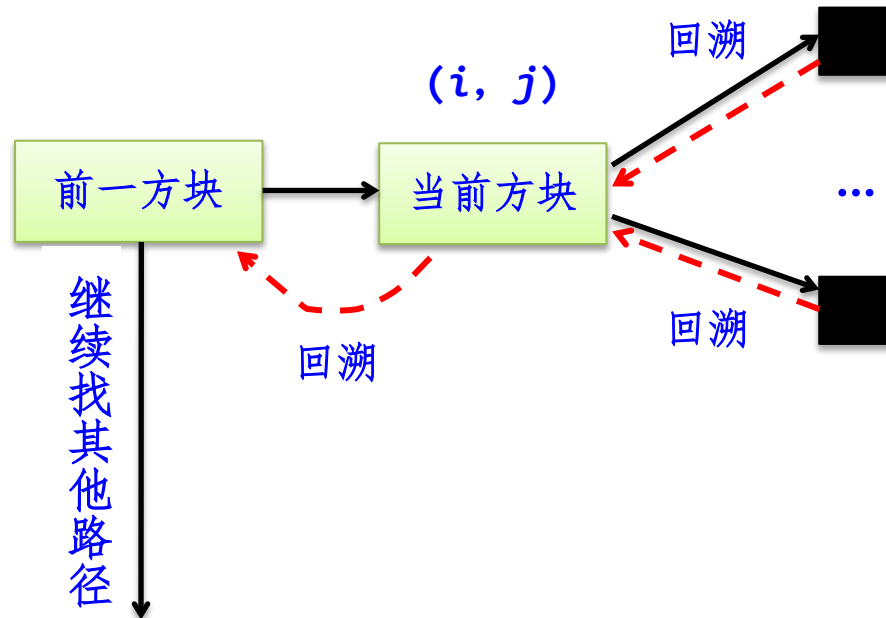


```
int dx[]={-1,0,1,0};  
int dy[]={0,1,0,-1};
```

//x方向的偏移量

//y方向的偏移量

迷宫问题的搜索过程

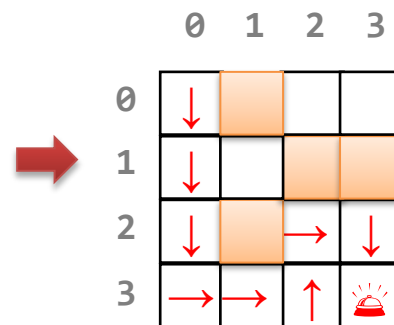
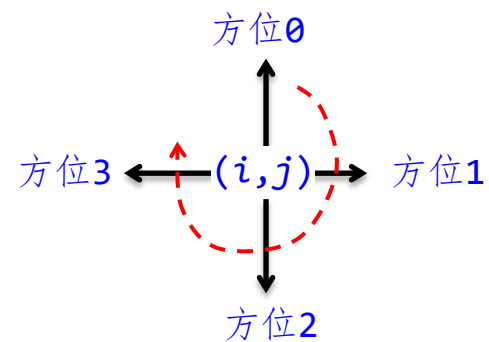
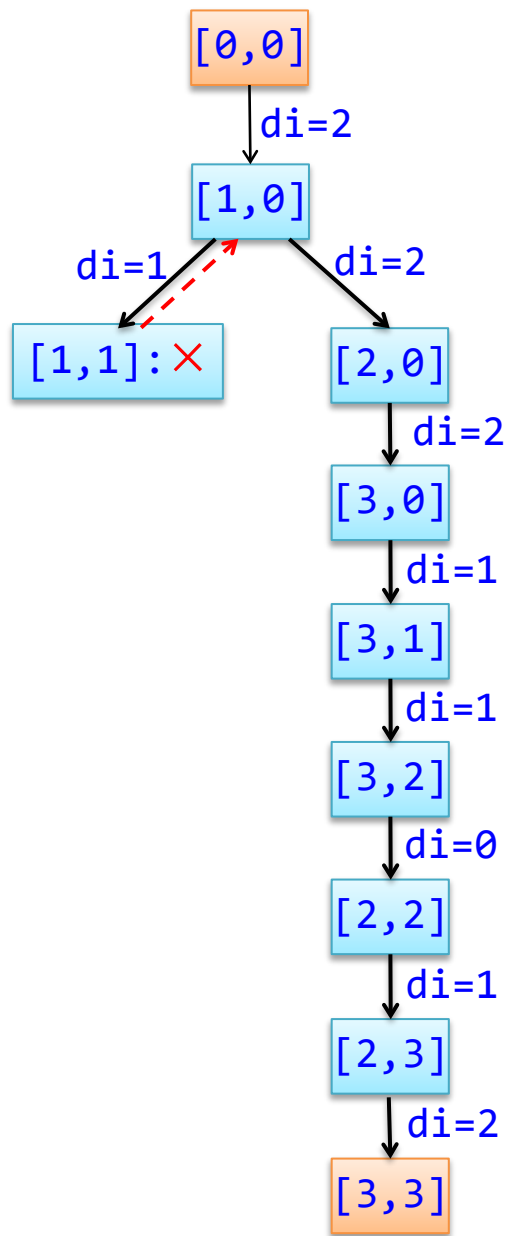
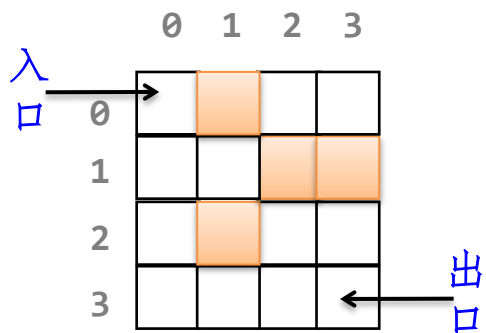


- 用栈记录走过的路径
- 路径：由方块和方块之间的走向（方位）构成





```
struct Box                                //方块类型
{  int i;                                //方块的行号
   int j;                                //方块的列号
   int di;                               //di是下一个相邻可走方位的方位号
   Box() {}                             //构造函数
   Box(int i1,int j1,int d1):           //重载构造函数
       i(i1),j(j1),di(d1) {}
};
```



```

bool mgpath(int xi,int yi,int xe,int ye)
//求一条从(xi,yi)到(xe,ye)的迷宫路径
{  int i,j,di,i1,j1;
    bool find;
    Box b,b1;
    stack<Box> st;
    b=Box(xi,yi,-1);
    st.push(b);
    mg[xi][yi]=-1;

    while (!st.empty())
    {  b=st.top();
        if (b.i==xe && b.j==ye)
        {  disppath(st);
            return true;
        }
    }
}

```

//建立一个栈

//入口方块进栈

//为避免来回找相邻方块置mg值为-1

//栈不空时循环

//取栈顶方块,称为当前方块

//输出栈中所有方块构成一条路径

//找到一条路径后返回true

find=false;	//否则继续找路径
di=b.di;	
while (di<3 && find==false)	//找b的一个相邻可走方块
{ di++;	//找下一个方位的相邻方块
i=b.i+dx[di];	//找b的di方位的相邻方块(i,j)
j=b.j+dy[di];	
if (i>=0 && i<m && j>=0 && j<n && mg[i][j]==0)	
find=true;	//(i,j)方块有效且可走
}	
if (find)	//找到一个相邻可走方块(i,j)
{ st.top().di=di;	//修改栈顶方块的di为新值
b1=Box(i,j,-1);	//建立相邻可走方块(i,j)的对象b1
st.push(b1);	//b1进栈
mg[i][j]=-1;	//为避免来回找相邻方块置mg值为-1
}	
else	//栈顶方块没有找到任何相邻可走方块
{ mg[b.i][b.j]=0;	//恢复栈顶方块的迷宫值
st.pop();	//将栈顶方块退栈
}	
}	
return false;	//没有找到迷宫路径,返回false
}	

```

void disppath(stack<Box>& st) //输出栈中所有方块构成一条迷宫路径
{
    Box b;
    vector<Box> apath; //存放一条迷宫路径
    while (!st.empty()) //出栈所有的方块
    {
        b=st.top(); st.pop();
        apath.push_back(b);
    }
    reverse(apath.begin(),apath.end()); //逆置apath(也可反向输出apath)
    cout << "一条迷宫路径: ";
    for (int i=0;i<apath.size();i++)
        cout << "[" << apath[i].i << "," << apath[i].j << "]" ";
    cout << endl;
}

```


设计主程序

```
int main()
{   int xi=0,yi=0,x=3,y=3;
    printf("求(%d,%d)到(%d,%d)的迷宫路径\n",xi,yi,x,y);
    if (!mgpath(xi,yi,x,y))
        cout << "不存在迷宫路径\n";
    return 0;
}
```

程序
验证



F:\C++\C++代码\ch3\Maze1.exe

求<0,0>到<3,3>的迷宫路径
一条迷宫路径: [0,0] [1,0] [2,0] [3,0] [3,1] [3,2] [2,2] [2,3] [3,3]



	0	1	2	3
0	↓			
1	↓			
2	↓		→	↓
3	→	→	↑	⊙



- 为什么找到的路径不一定是最短路径？
- 如何求所有的迷宫路径？